CWE-707: Improper Neutralization

Weakness ID: 707
Vulnerability Mapping: DISCOURAGED
Abstraction: Pillar

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product does not ensure or incorrectly ensures that structured messages or data are well-formed and that certain security properties are met before being read from an upstream component or sent to a downstream component.

Extended Description

If a message is malformed, it may cause the message to be incorrectly interpreted.

Neutralization is an abstract term for any technique that ensures that input (and output) conforms with expectations and is "safe." This can be done by:

- checking that the input/output is already "safe" (e.g. validation)
- transformation of the input/output to be "safe" using techniques such as filtering, encoding/decoding, escaping/unescaping, quoting/unquoting, or canonicalization
- preventing the input/output from being directly provided by an attacker (e.g. "indirect selection" that maps externally-provided values to internally-controlled values)
- preventing the input/output from being processed at all

This weakness typically applies in cases where the product prepares a control message that another process must act on, such as a command or query, and malicious input that was intended as data, can enter the control plane instead. However, this weakness also applies to more general cases where there are not always control implications.

Common Consequences

Impact	Details
Other	**Scope: Other**

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
MemberOf	V	1000	Research Concepts
ParentOf	C	20	Improper Input Validation
ParentOf	C	74	Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection')
ParentOf	C	116	Improper Encoding or Escaping of Output
ParentOf	C	138	Improper Neutralization of Special Elements
ParentOf	B	170	Improper Null Termination
ParentOf	C	172	Encoding Error
ParentOf	B	182	Collapse of Data into Unsafe Value
ParentOf	C	228	Improper Handling of Syntactically Invalid Structure
ParentOf	B	240	Improper Handling of Inconsistent Structural Elements
ParentOf	B	463	Deletion of Data Structure Sentinel
ParentOf	B	1426	Improper Validation of Generative AI Output

Relevant to the view "Architectural Concepts" (View-1008)

Nature	Type	ID	Name
MemberOf	C	1020	Verify Message Integrity

Modes Of Introduction

Phase	Note
Implementation	REALIZATION: This weakness is caused during implementation of an architectural security tactic.

Applicable Platforms

Languages	Class: Not Language-Specific *(Undetermined Prevalence)*
Operating Systems	Class: Not OS-Specific *(Undetermined Prevalence)*
Architectures	Class: Not Architecture-Specific *(Undetermined Prevalence)*
Technologies	Class: Not Technology-Specific *(Undetermined Prevalence)*

Weakness Ordinalities

Ordinality	Description
Primary	*(where the weakness exists independent of other weaknesses)*

Detection Methods

Method	Details
Automated Static Analysis	Automated static analysis, commonly referred to as Static Application Security Testing (SAST), can find some instances of this weakness by analyzing source code (or binary/compiled code) without having to execute it. Typically, this is done by building a model of data flow and control flow, then searching for potentially-vulnerable patterns that connect "sources" (origins of input) with "sinks" (destinations where the data interacts with external components, a lower layer such as the OS, etc.)

Memberships

Nature	Type	ID	Name
MemberOf	C	990	SFP Secondary Cluster: Tainted Input to Command
MemberOf	C	1370	ICS Supply Chain: Common Mode Frailties
MemberOf	C	1407	Comprehensive Categorization: Improper Neutralization

Vulnerability Mapping Notes

Usage	**DISCOURAGED** *(this CWE ID should not be used to map to real-world vulnerabilities)*
Reason	Abstraction
Rationale	This CWE entry is extremely high-level, a Pillar.
Comments	Consider children or descendants of this entry instead.

Notes

Maintenance

Concepts such as validation, data transformation, and neutralization are being refined, so relationships between CWE-20 and other entries such as CWE-707 may change in future versions, along with an update to the Vulnerability Theory document.

Related Attack Patterns

CAPEC-ID	Attack Pattern Name
CAPEC-250	XML Injection
CAPEC-276	Inter-component Protocol Manipulation
CAPEC-277	Data Interchange Protocol Manipulation
CAPEC-278	Web Services Protocol Manipulation

CAPEC-279	SOAP Manipulation
CAPEC-3	Using Leading 'Ghost' Character Sequences to Bypass Input Filters
CAPEC-43	Exploiting Multiple Input Interpretation Layers
CAPEC-468	Generic Cross-Browser Cross-Domain Theft
CAPEC-52	Embedding NULL Bytes
CAPEC-53	Postfix, Null Terminate, and Backslash
CAPEC-64	Using Slashes and URL Encoding Combined to Bypass Validation Logic
CAPEC-7	Blind SQL Injection
CAPEC-78	Using Escaped Slashes in Alternate Encoding
CAPEC-79	Using Slashes in Alternate Encoding
CAPEC-83	XPath Injection
CAPEC-84	XQuery Injection

Content History

Submissions

Submission Date	Submitter	Organization
2008-09-09 *(CWE 1.0, 2008-09-09)*	CWE Content Team	MITRE
	Note: this date reflects when the entry was first published. Draft versions of this entry were provided to members of the CWE community and modified between Draft 9 and 1.0.	

Modifications

Previous Entry Names